

1교시: Git 이론과 변경 이력 모델

Week 3 Day 3 Lesson 1

서비스 분리와 데이터 의존성

데이터 구조와 경계를 어떻게 설계하느냐가 서비스의 독립성과 안정성을 결정합니다.

서비스별 데이터 책임 (권장)

각 서비스가 자신의 데이터를 소유하고 관리

주문 서비스
주문 도메인 로직

결제 서비스
결제 도메인 로직

배송 서비스
배송 도메인 로직

주문 DB

결제 DB

배송 DB

- ✓ 각 서비스가 자신의 데이터에 대한 완전한 소유권을 가짐
- ✓ 스키마 변경이 다른 서비스에 영향 없음
- ✓ 장애가 발생해도 영향 범위가 해당 서비스 내부로 제한

공유 DB 위험 (비권장)

여러 서비스가 하나의 DB를 공유

주문 서비스
주문 도메인 로직

결제 서비스
결제 도메인 로직

배송 서비스
배송 도메인 로직

공유 DB

- ⚠ 스키마 변경 시 모든 서비스에 영향
- ⚠ 한 서비스의 장애가 전체에 전파될 수 있음
- ⚠ 데이터 접근 경계가 모호해짐
- ⚠ 서비스 간 강한 결합 발생
- ⚠ 독립적 배포와 확장이 어려움

각 서비스가 자신 데이터 완전 소유	data ownership (데이터 소유권)	소유권이 불명확하고 책임 소재 모호
독립적 스키마 관리, 변경 영향 최소화	schema coupling (스키마 결합도)	공유 스키마로 변경 시 다수 서비스 영향
장애 영향 범위가 서비스 내부로 제한	장애 영향 범위 (blast radius)	하나의 장애가 전체 시스템으로 확산
명확한 접근 경계로 보안 및 규정 준수 용이	접근 경계 (access boundary)	접근 경계 모호, 불필요한 데이터 노출 위험

핵심 원칙

- ✓ 서비스는 데이터를 소유해야 한다
- ✓ 데이터 경계가 서비스 경계다
- ✓ 독립적 변경, 배포, 확장을 가능하게 한다
- ✓ 작은 변경이 큰 장애로 번지지 않게 한다

💡 좋은 데이터 설계는 단순한 테이블 분리가 아니라, 서비스의 자율성, 안정성, 확장성을 보장하는 핵심 전략입니다.

수업 목표

- Git이 파일 백업 도구가 아니라 변경 이력 그래프임을 이해한다.
- working tree, staging area, commit, branch, remote, tag를 구분한다.
- GitHub 협업과 GitHub Actions가 Git 이력 위에서 동작한다는 점을 설명한다.

Git의 핵심 모델

working tree -> staging area -> commit -> branch -> remote

개념	의미
working tree	현재 파일 상태
staging area	다음 commit에 포함할 변경
commit	변경 스냅샷과 메시지
branch	commit을 가리키는 이름
remote	GitHub 같은 원격 저장소
tag	특정 commit에 붙이는 고정 이름

실습

```
cd /mnt/d/paperclip
git status -sb
git branch --show-current
git log --oneline -5
git remote -v
```

해석 기준

출력	해석
M	tracked file 수정
??	untracked file
main	현재 branch
origin	기본 원격 저장소

왜 인프라 엔지니어에게 Git이 중요한가

인프라 변경도 코드 변경처럼 추적되어야 한다.

변경	Git이 필요한 이유
Terraform module 수정	누가 어떤 리소스를 바꿨는지 추적
Kubernetes manifest 수정	배포된 설정과 commit 연결
GitHub Actions workflow 수정	CI/CD 동작 변경 추적
Secret 참조 변경	민감정보 노출 없이 사용 이력 관리

핵심 포인트

Git 이력을 이해하지 못하면 GitHub Actions와 배포 자동화도 흔들린다.

```
무엇을 바꿨는가
누가 바꿨는가
언제 바꿨는가
어떤 commit을 배포했는가
```

Evidence Note

```
# W3D3S1 Git Model
```

- current branch:
- modified files:
- remote:
- latest commit:
- tag use case: